How Backpropagation improves neural network accuracy during training

Name:	**ALI WERRAD KHAN**
Roll number:	**24152502**
GitHub repository link:	ak25adm/Machine-Learning-and-Neural-Networks: How Backpropagation improves Neural Network Accuracy During Training

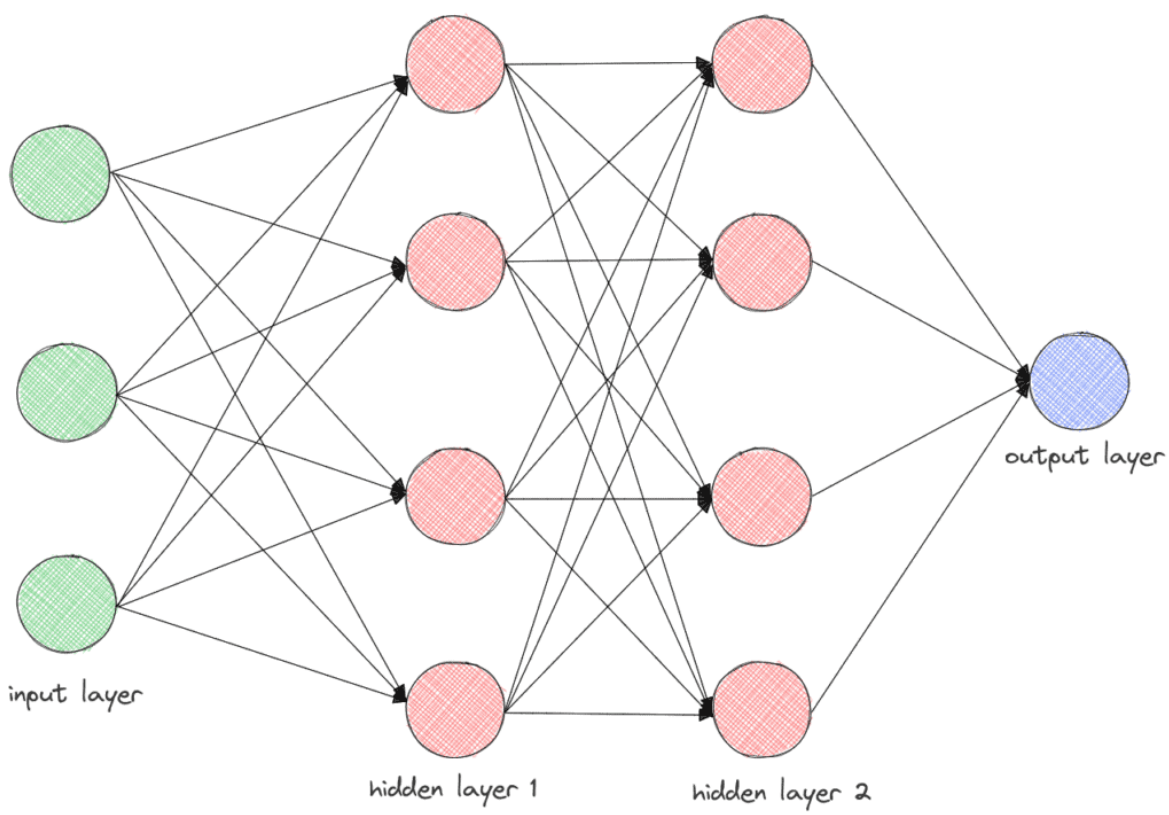
Neural Network

A neural network is a computational model, and inspired by the structure of human brain. It is composed of units called neurons. These neurons work together to process information and learn patterns from data.

The basic structure of a neural network consists of following layers:

- An input layer
- One or more hidden layers
- An output layer.

Each layer contains multiple neurons, and each neuron is connected to neurons in the next layer. (Three-Layered Neural Network, n.d.)

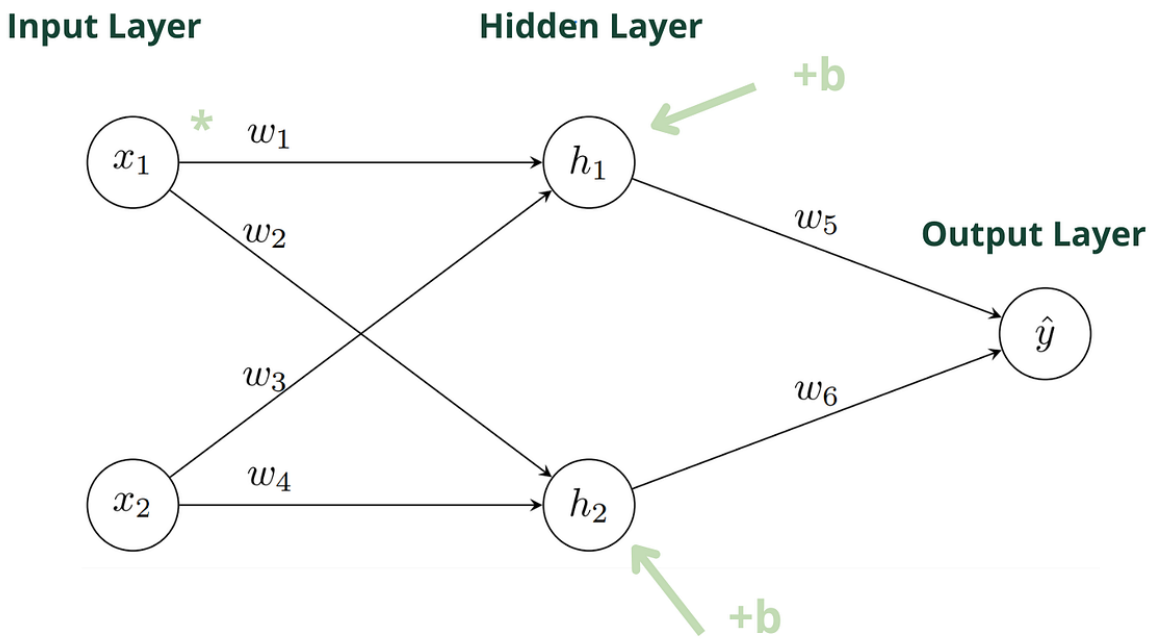


How Neural Networks Work:

A neuron in a neural network is a decision-maker. It receives inputs, and multiplies each input by a weight. The weight shows how important that input is (Lee, What is a neural network?,

n.d.). Then, the neuron adds a bias, which is a number that the model learns. Think of the bias as a way to shift the output up or down (Weights and Bias in Neural Networks, 2025).

After this, the neuron passes the result through an activation function. This function helps the neuron decide what the final output should be.



Forward propagation:

Forward propagation is the process by which a neural network makes a prediction. Input data is given to model. This data then moves through hidden layers (Salve, 2024).

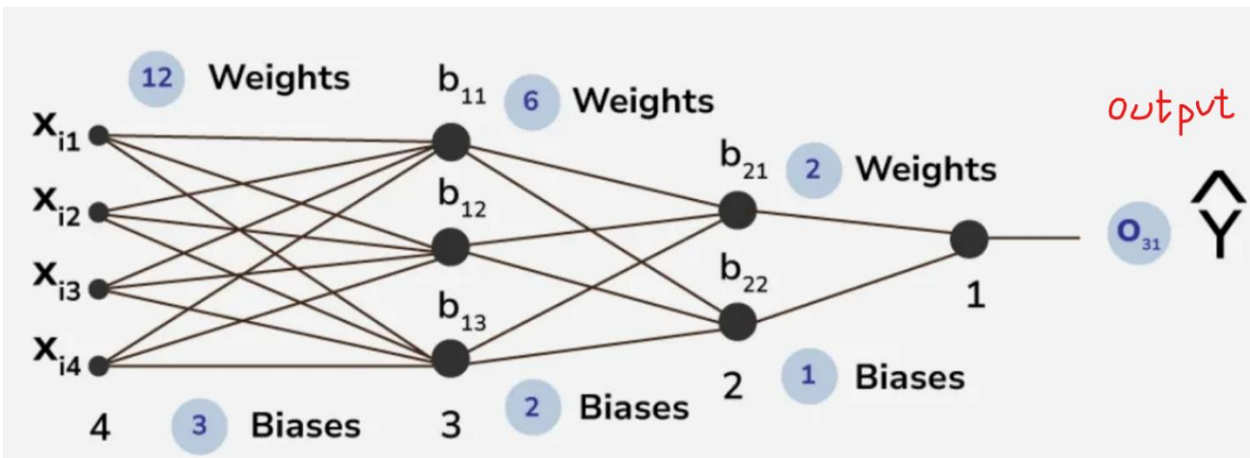
At each neuron, the network:

1. Calculates the weighted sum of the inputs (weights show the importance of each input).
2. Adds a bias.
3. Applies an activation function.

$$z = \sum(w_i x_i) + b$$

$$a = f(z)$$

- w_i is weight, x_i is input, b is bias, and f is the activation function.



The output from one layer becomes the input for the next layer. This continues until the data reaches the output layer, where the network gives its final prediction (Divakar, 2024).

Backward Propagation

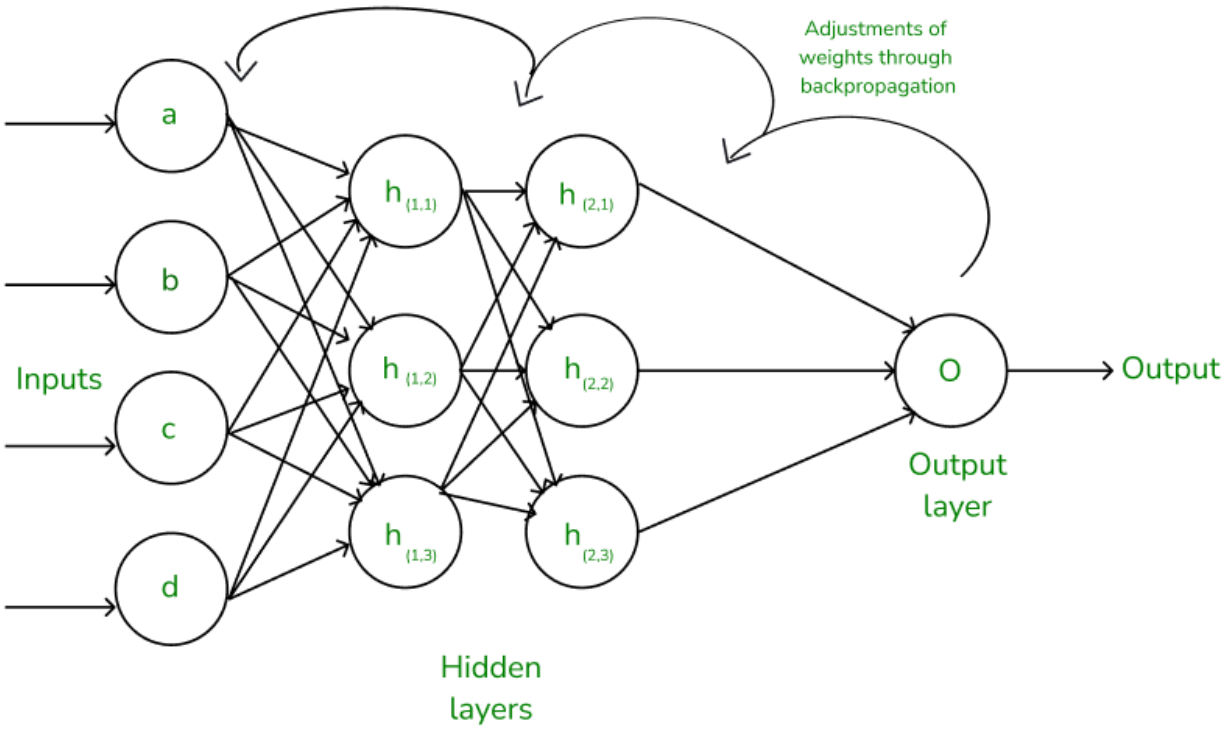
After the network makes a prediction using forward propagation, we compare its output with the actual correct answer. This tells us how wrong the prediction is.

Backpropagation works by figuring out how much each weight and bias contributed to the error. It does this by moving backward through the network, and calculating something called gradients for each weight (Salve, 2024). Gradients show the direction and size of the change needed to reduce the error.

Once we have the gradients, the network updates its weights and biases (Brownlee, 2021).

In simple terms:

- Forward propagation → network makes a guess
- Backpropagation → figure out how to adjust weights to make better guesses next time

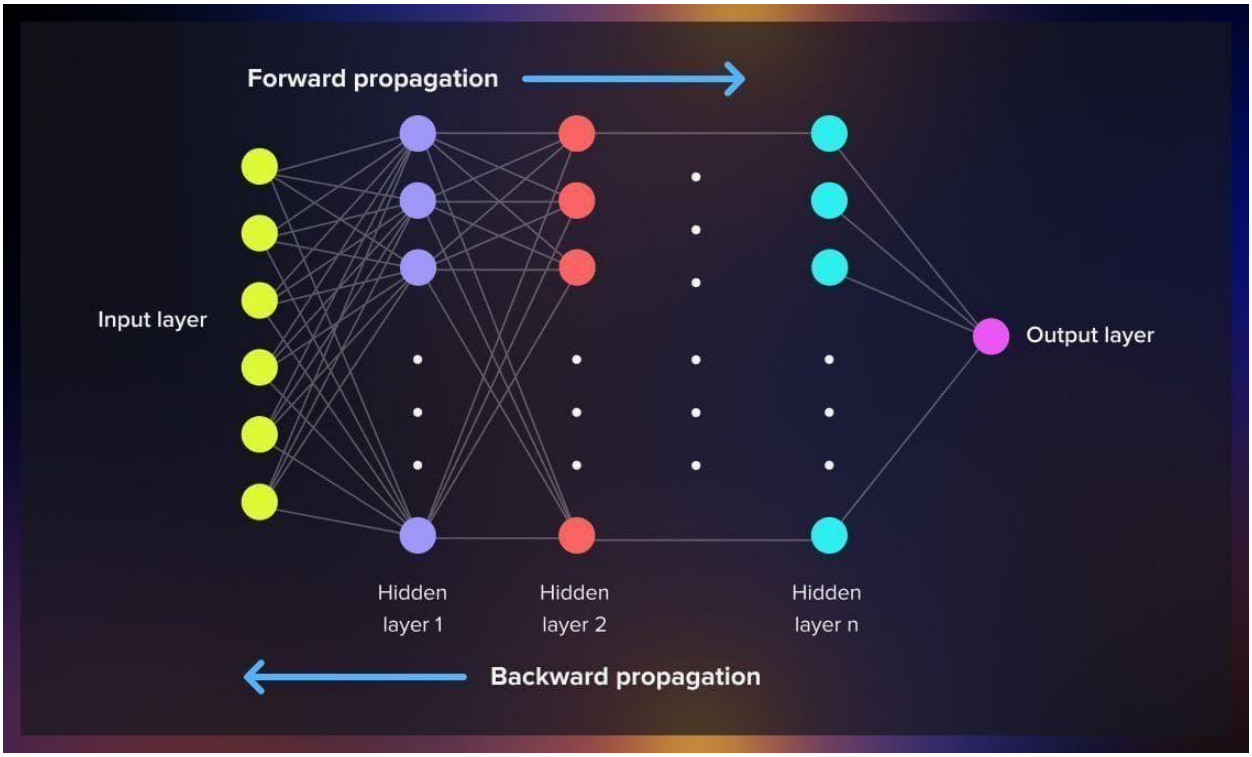


Neural Networks: Complete Overview

Each neuron uses weights and biases (to understand the importance of inputs and errors).

- Forward propagation is how the network makes predictions.
- Backward propagation is how the network learns from its mistakes and reduces error.

These processes allow neural networks to learn from data and make accurate predictions.



Backpropagation

Backpropagation tells each neuron how it should adjust itself to perform better next time (Backpropagation in Neural Network, 2026).

Imagine you are making a dish, and you have 10 cooks. Each cook is responsible for a specific task. Together, they prepare the final dish. After tasting the dish, you realize something is wrong. Maybe it is too salty. Instead of blaming the whole team, you try to figure out which cook made the mistake. The cook responsible for adding salt made the biggest mistake. Over time, with repeated feedback and improvement, the cooks start working better and the dish becomes perfect.

This is exactly how backpropagation works. The final dish is the output. The taste difference is error. You analyze the error and send feedback to each cook based on their role. Some cooks get more feedback, others less. Neurons continuously update their weights based on feedback from backpropagation.

Crux of Backpropagation

A neuron takes inputs, multiplies them by weights, adds a bias, and passes the result through an activation function (Weights and Bias in Neural Networks, 2025). **The output of one neuron becomes the input for the next layer.** Prediction is not always correct in forward propagation. The difference between the predicted output and the actual value is called the **error or loss** (Yathish, 2022).

Then, network sends the error backward from the output layer to the input layer. This is done using a method called the chain rule (Chain Rule & Backpropagation, n.d.), you can think of it as tracing responsibility.

Then, network updates the weights. The update is usually done using a rule like:

*new weight = old weight - learning rate * gradient*

- A **gradient** is the slope of loss function used to guide weight updates in learning.
- The **learning rate** determines how big a step is taken when updating weights.

Backpropagation ensures that the network becomes smarter with each iteration.

$$w_{new} = w_{old} - \eta \times \left(\frac{\partial L}{\partial w}\right)$$

- η is the learning rate and $\frac{\partial L}{\partial w}$ is the gradient of the loss with respect to the weight.

Datasets

In this project, two different datasets are used to demonstrate effect of backpropagation.

The first dataset is the diabetes dataset. The goal of this dataset is to predict whether a person has diabetes or not. This is a binary classification problem. The dataset includes several features such as:

- Glucose level
- Blood pressure
- Insulin level
- Body mass index (BMI)
- Age
- Number of pregnancies

	A	B	C	D	E	F	G	H	I	J
1	**Pregnancies**	**Glucose**	**BloodPressure**	**SkinThickness**	**Insulin**	**BMI**	**DiabetesPedigreeFunction**	**Age**	**Outcome**	
2	6	148	72	35	0	33.6	0.627	50	1	
3	1	85	66	29	0	26.6	0.351	31	0	
4	8	183	64	0	0	23.3	0.672	32	1	
5	1	89	66	23	94	28.1	0.167	21	0	
6	0	137	40	35	168	43.1	2.288	33	1	
7	5	116	74	0	0	25.6	0.201	30	0	
8	3	78	50	32	88	31	0.248	26	1	
9	10	115	0	0	0	35.3	0.134	29	0	
10	2	197	70	45	543	30.5	0.158	53	1	
11	8	125	96	0	0	0	0.232	54	1	
12	4	110	92	0	0	37.6	0.191	30	0	
13	10	168	74	0	0	38	0.537	34	1	
14	10	139	80	0	0	27.1	1.441	57	0	
15	1	189	60	23	846	30.1	0.398	59	1	
16	5	166	72	19	175	25.8	0.587	51	1	
17	7	100	0	0	0	30	0.484	32	1	
18	0	118	84	47	230	45.8	0.551	31	1	
19	7	107	74	0	0	29.6	0.254	31	1	
20	1	103	30	38	83	43.3	0.183	33	0	
21	1	115	70	30	96	34.6	0.529	32	1	
22	3	126	88	41	235	39.3	0.704	27	0	
23	8	99	84	0	0	35.4	0.388	50	0	
24	7	196	90	0	0	39.8	0.451	41	1	
25	9	119	80	35	0	29	0.263	29	1	
26	11	143	94	33	146	36.6	0.254	51	1	
27	10	125	70	26	115	31.1	0.205	41	1	
28	7	147	76	0	0	39.4	0.257	43	1	

These features act as inputs to the neural network. The label (Label: what we will predict afterwards) is the “Outcome” column, which tells whether the patient is diabetic or not (Luca, 2025). During training, the model learns patterns between these health-related features and the final outcome (how features effect the person to be diabetic or not).

The second dataset is the car price dataset. In this case, the goal is to estimate the price of a car. The dataset contains features such as

- Transmission type
- Fuel type
- Number of seats
- Engine capacity
- Ownership history
- Kilometers driven

	A	B	C	D	E	F	G	H	I	J	K
1		car_name	car_prices_in_rupee	kms_driven	fuel_type	transmission	ownership	manufacture	engine	Seats	
2	0	Jeep Compass 2.0 Longitude Option BSIV	10.03 Lakh	86,226 kms	Diesel	Manual	1st Owner	2017	1956 cc	5 Seats	
3	1	Renault Duster RXZ Turbo CVT	12.83 Lakh	13,248 kms	Petrol	Automatic	1st Owner	2021	1330 cc	5 Seats	
4	2	Toyota Camry 2.5 G	16.40 Lakh	60,343 kms	Petrol	Automatic	1st Owner	2016	2494 cc	5 Seats	
5	3	Honda Jazz VX CVT	7.77 Lakh	26,696 kms	Petrol	Automatic	1st Owner	2018	1199 cc	5 Seats	
6	4	Volkswagen Polo 1.2 MPI Highline	5.15 Lakh	69,414 kms	Petrol	Manual	1st Owner	2016	1199 cc	5 Seats	
7	5	Volkswagen Vento 1.2 TSI Highline AT	7.66 Lakh	49,719 kms	Petrol	Automatic	1st Owner	2017	1197 cc	5 Seats	
8	6	Volkswagen Vento 1.2 TSI Highline Plus AT	7.58 Lakh	43,688 kms	Petrol	Automatic	1st Owner	2017	1197 cc	5 Seats	
9	7	Honda WR-V VX Diesel	11.60 Lakh	14,470 kms	Diesel	Manual	1st Owner	2021	1498 cc	5 Seats	
10	8	Honda City i VTEC CVT SV	6.99 Lakh	21,429 kms	Petrol	Automatic	1st Owner	2015	1497 cc	5 Seats	
11	9	Renault Duster Petrol RXS CVT	7.53 Lakh	31,750 kms	Petrol	Automatic	1st Owner	2017	1498 cc	5 Seats	
12	10	Maruti Baleno 1.2 Alpha	6.43 Lakh	38,203 kms	Petrol	Manual	1st Owner	2017	1197 cc	5 Seats	
13	11	Honda City i VTEC CVT SV	5.43 Lakh	1,10,284 kms	Petrol	Automatic	1st Owner	2014	1497 cc	5 Seats	
14	12	Mahindra XUV300 W6	8.62 Lakh	10,381 kms	Petrol	Manual	1st Owner	2020	1197 cc	5 Seats	
15	13	Jeep Compass 1.4 Limited Plus BSIV	16.78 Lakh	32,378 kms	Petrol	Automatic	1st Owner	2019	1368 cc	5 Seats	
16	14	Honda City V MT	10.03 Lakh	38,906 kms	Petrol	Manual	1st Owner	2020	1498 cc	5 Seats	
17	15	Hyundai Grand i10 AT Asta	5.63 Lakh	59,313 kms	Petrol	Automatic	2nd Owner	2016	1197 cc	5 Seats	
18	16	Hyundai i20 1.4 Asta	6.67 Lakh	85,672 kms	Diesel	Manual	1st Owner	2017	1396 cc	5 Seats	
19	17	Maruti Ciaz 1.4 Delta	6.73 Lakh	34,971 kms	Petrol	Manual	2nd Owner	2017	1373 cc	5 Seats	
20	18	Nissan Micra XL Optional	3.21 Lakh	35,894 kms	Petrol	Manual	2nd Owner	2015	1198 cc	5 Seats	
21	19	Maruti Ciaz Alpha Automatic BSIV	8.50 Lakh	56,000 kms	Petrol	Automatic	1st Owner	2019	1462 cc	5 Seats	
22	20	Maruti Swift Dzire VXI	4.34 Lakh	56,568 kms	Petrol	Manual	1st Owner	2014	1197 cc	5 Seats	
23	21	Renault KWID RXT	3.11 Lakh	48,872 kms	Petrol	Manual	1st Owner	2017	799 cc	5 Seats	
24	22	Renault KWID RXL BSIV	3.70 Lakh	17,346 kms	Petrol	Manual	1st Owner	2019	799 cc	5 Seats	
25	23	Jeep Compass 2.0 Limited Plus BSIV	15.88 Lakh	15,414 kms	Diesel	Manual	1st Owner	2019	1956 cc	5 Seats	
26	24	Hyundai Grand i10 1.2 Kappa Magna BSIV	5.41 Lakh	21,239 kms	Petrol	Manual	1st Owner	2018	1197 cc	5 Seats	
27	25	Maruti Alto K10 LXI	2.23 Lakh	62,361 kms	Petrol	Manual	1st Owner	2014	998 cc	5 Seats	
28	26	Hyundai Verna 1.6 VTVT S Option	5.88 Lakh	79,329 kms	Petrol	Manual	2nd Owner	2015	1591 cc	5 Seats	

These features describe the characteristics of car and are used as inputs for the model. The label in this dataset is the “Price” of the car.

In summary, the diabetes dataset represents a classification problem, while the car price dataset represents a regression problem with a continuous outputs.

Test 1: Training Neural Network Model

Simple Neural Network

We applied a simple logistic regression model. It achieved an accuracy of 74%. Logistic regression is a ML algorithm used for classification tasks (Lee, What is logistic regression?, n.d.). Logistic regression is different from neural networks. It (logistic regression) does not involve multiple layers or complex structures.

Simple ML model

In [3]:
```
df = pd.read_csv('diabetes/diabetes.csv')
```

In [5]:
```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression(max_iter = 1000)
model.fit(X_train, y_train)
```

Out[5]:
```
LogisticRegression(max_iter=1000)
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [6]:
```
score = model.score(X_test, y_test)

print("Model Accuracy:", score)
```

```
Model Accuracy: 0.7467532467532467
```

Activate Windows
Go to Settings to activate W

Neural Network

Neural networks (as compared to simple ML models), can learn more complex patterns, which often leads to better performance.

The use of neural networks shows a clear improvement in performance as the number of epochs increases.

Initially, with only 3 epochs, the model achieves an accuracy of 0.7182.

- An **epoch** is one complete pass of the entire training dataset through the neural network during the training process.

3 epochs

In [28]:

```
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])


history = model.fit(X_train, y_train, epochs=3)
```

```
Epoch 1/3
20/20 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.5537 - loss: 0.7063
Epoch 2/3
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6678 - loss: 0.6327
Epoch 3/3
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7182 - loss: 0.5870
```

As training continues, the accuracy improves: 0.7818 at 10 epochs:

In [26]:

```
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])


history = model.fit(X_train, y_train, epochs=10)
```

```
Epoch 1/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.5619 - loss: 0.7042
Epoch 2/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6987 - loss: 0.6481
Epoch 3/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.7296 - loss: 0.6155
Epoch 4/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7427 - loss: 0.5831
Epoch 5/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7508 - loss: 0.5508
Epoch 6/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7736 - loss: 0.5215
Epoch 7/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7720 - loss: 0.4953
Epoch 8/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7785 - loss: 0.4757
Epoch 9/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7866 - loss: 0.4641
Epoch 10/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7818 - loss: 0.4582
```

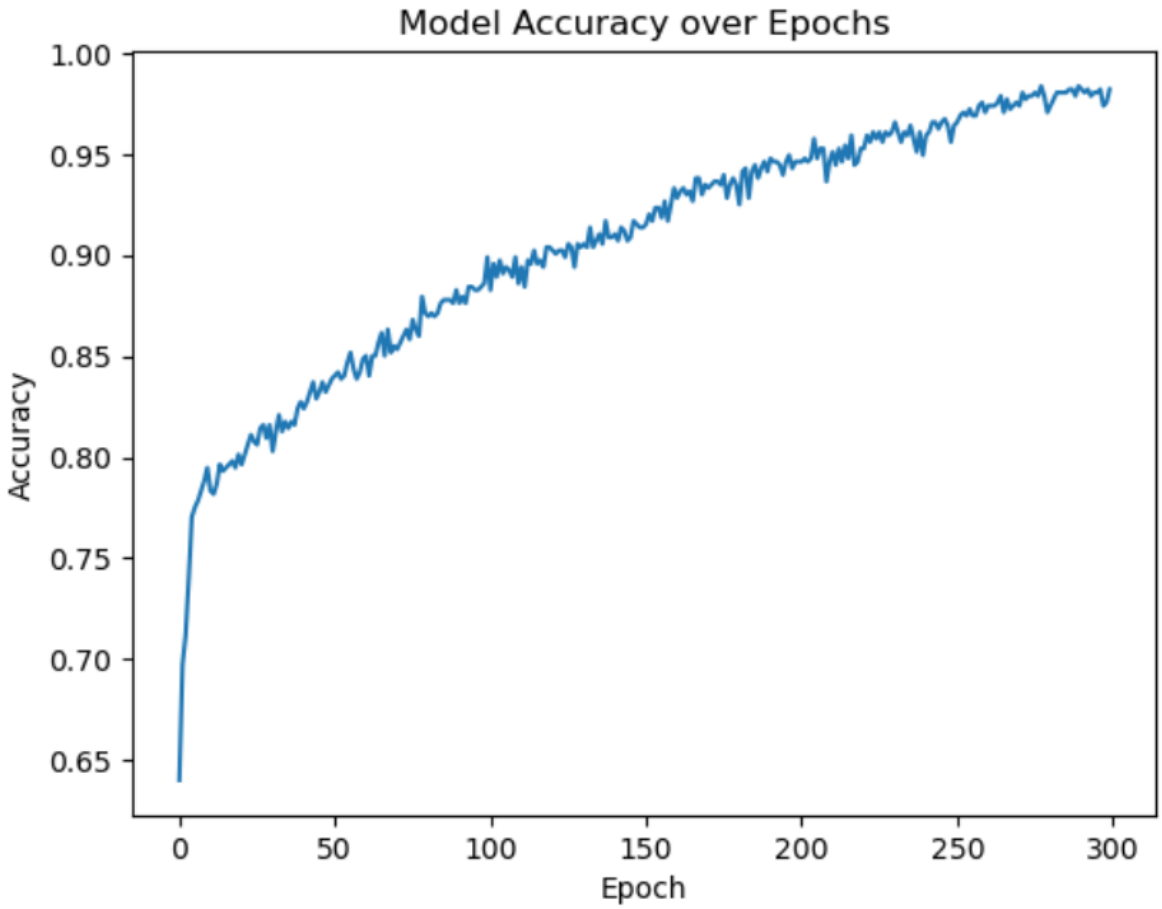
Notice how accuracy improve at every epoch.

0.8160 at 30 epochs, 0.8697 at 70 epochs, 0.9332 at 150 epochs, and finally reaching **0.9821 (98% accuracy)** at 300 epochs. This demonstrates that neural networks become more accurate with more training.

```
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9821 - loss: 0.0708
Epoch 289/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9788 - loss: 0.0712
Epoch 290/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9837 - loss: 0.0674
Epoch 291/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9821 - loss: 0.0667
Epoch 292/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9805 - loss: 0.0672
Epoch 293/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9821 - loss: 0.0637
Epoch 294/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9788 - loss: 0.0706
Epoch 295/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9805 - loss: 0.0702
Epoch 296/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9805 - loss: 0.0666
Epoch 297/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9821 - loss: 0.0635
Epoch 298/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9739 - loss: 0.0744
Epoch 299/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9756 - loss: 0.0672
Epoch 300/300
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9821 - loss: 0.0642
```

The reason behind this improvement is backpropagation. After each epoch, the neural network calculates the error (baeldung, 2025). Backpropagation sends this error backward through the network. Based on this information, the weights of the neurons are updated to reduce the error in the next iteration.

Accuracy graph of neural network with 300 epochs:



With more epochs, the network has more opportunities to adjust its weights (Backpropagation in Neural Network, 2026). Over time, model capture more complex relationships in the data.

Therefore, the significant increase in accuracy highlights the importance of backpropagation.

Test 2: Training Neural Network Model

Second dataset is a regression problem where the goal is to predict a continuous value. We evaluate the model using Mean Absolute Error (MAE) rather than accuracy.

Mean Absolute Error (MAE) measures the average magnitude of errors between the predicted values and the actual values (Mean Absolute Error In Machine Learning: What You Need To Know, 2023).

In simple terms, MAE tells us how far off our predictions are from the actual values.

Initially, we will use a simple network with 8 hidden layers and 1 output layers. There are 20, 30, 50, 50, 50, 30, 20, 10 neurons in respective layers. We are change these numbers in surely.

```
model = Sequential()

# Input layer + 1st hidden layer
model.add(Dense(20, activation='relu', input_dim=X_train.shape[1]))

# 2nd hidden layer
model.add(Dense(30, activation='relu'))

# 3rd hidden layer
model.add(Dense(50, activation='relu'))

# 4th hidden layer
model.add(Dense(50, activation='relu'))

# 5th hidden layer
model.add(Dense(50, activation='relu'))

model.add(Dense(30, activation='relu'))

model.add(Dense(20, activation='relu'))

model.add(Dense(10, activation='relu'))

# Output layer (1 neuron for regression)
model.add(Dense(1))

# Compile model
model.compile(
    optimizer='adam',
    loss='mean_squared_error',  # regression loss
    metrics=['mean_absolute_error']  # meaningful metric for regression
)

# Train the model
history = model.fit(X_train, y_train, epochs=100, validation_split=0.2)
```

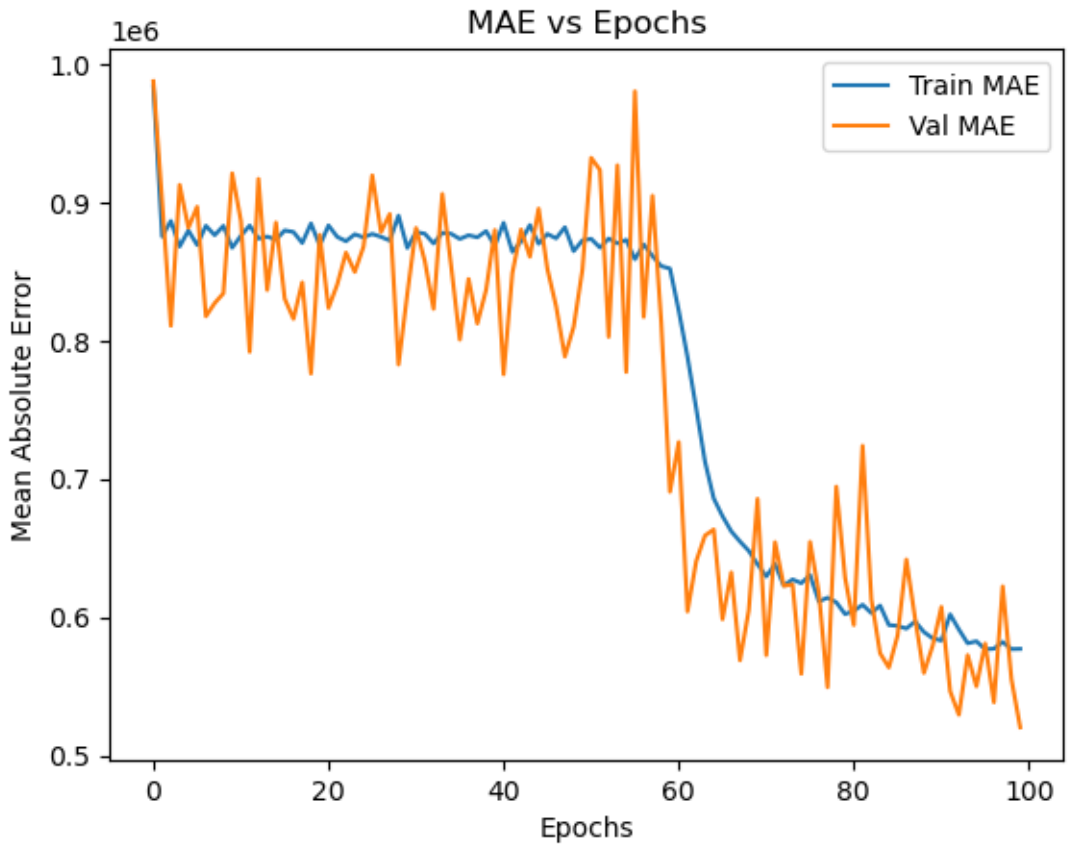
The model shows extremely high loss and mean absolute error (MAE). At epoch 10, MAE is around 785,000, indicating that the network has not yet learned the patterns effectively.

```
107/107 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 2930023858176.0000 - mean_absolute_error: 1017497.6875 - val_loss: 2449252286464.0000 - val_mea
n_absolute_error: 797681.0000
Epoch 2/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2537804267520.0000 - mean_absolute_error: 890013.5000 - val_loss: 2345478914048.0000 - val_mean
_absolute_error: 835549.1250
Epoch 3/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2507470798848.0000 - mean_absolute_error: 888885.5000 - val_loss: 2304694812672.0000 - val_mean
_absolute_error: 805401.2500
Epoch 4/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2448335044608.0000 - mean_absolute_error: 879248.0000 - val_loss: 2226910920704.0000 - val_mean
_absolute_error: 805492.3125
Epoch 5/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2301314465792.0000 - mean_absolute_error: 844705.4375 - val_loss: 2032167682048.0000 - val_mean
_absolute_error: 816041.8125
Epoch 6/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1855874662400.0000 - mean_absolute_error: 792133.6250 - val_loss: 1584726933504.0000 - val_mean
_absolute_error: 793976.2500
Epoch 7/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1611422498816.0000 - mean_absolute_error: 792208.9375 - val_loss: 1580091047936.0000 - val_mean
_absolute_error: 790660.8125
Epoch 8/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1600031686656.0000 - mean_absolute_error: 790569.9375 - val_loss: 1565153427456.0000 - val_mean
_absolute_error: 723141.1250
Epoch 9/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1601546878976.0000 - mean_absolute_error: 788982.6250 - val_loss: 1550314373120.0000 - val_mean
_absolute_error: 752449.1250
Epoch 10/10
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1596894871552.0000 - mean_absolute_error: 785951.3125 - val_loss: 1596259172352.0000 - val_mean
_absolute_error: 850819.8750
```

By 100 epochs, MAE drops to about 577,000. Weights are updated through repeated forward and backward passes. Each epoch allows the network to compare predictions to actual values, calculate errors, and adjust weights to reduce these errors.

```
Epoch 92/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1061672058880.0000 - mean_absolute_error: 602066.0000 - val_loss: 1031927169024.0000 - val_mean
_absolute_error: 546455.0000
Epoch 93/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1034829758464.0000 - mean_absolute_error: 591107.7500 - val_loss: 1065613918208.0000 - val_mean
_absolute_error: 529451.8750
Epoch 94/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1009984995328.0000 - mean_absolute_error: 581161.3125 - val_loss: 1019761983488.0000 - val_mean
_absolute_error: 572655.5000
Epoch 95/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1011909132288.0000 - mean_absolute_error: 582605.1875 - val_loss: 999729463296.0000 - val_mean_
absolute_error: 550014.8750
Epoch 96/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 992837632000.0000 - mean_absolute_error: 576791.7500 - val_loss: 1015913840640.0000 - val_mean_
absolute_error: 581011.5000
Epoch 97/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 998802391040.0000 - mean_absolute_error: 577243.8125 - val_loss: 990435147776.0000 - val_mean_a
bsolute_error: 538499.8125
Epoch 98/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1024549322752.0000 - mean_absolute_error: 582098.0000 - val_loss: 1027737911296.0000 - val_mean
_absolute_error: 622315.3125
Epoch 99/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 979981828096.0000 - mean_absolute_error: 576941.7500 - val_loss: 1013650423808.0000 - val_mean_
absolute_error: 555149.5625
Epoch 100/100
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1001564995584.0000 - mean_absolute_error: 577109.1875 - val_loss: 1040561930240.0000 - val_mean
_absolute_error: 520445.6250
```

Epochs and MAE graph:



- This graph basically shows 2 lines.
- Orange line shows MAE of data on which neural network test itself.
- Blue line shows MAE of data on which neural network is trained.
- Both lines are close to each other, which shows a strong neural network.

Increasing the number of epochs from 100 to 300 significantly reduces MAE from 577,000 to 530,000. More epochs mean more cycles of error correction, allowing the network to fine-tune its weights and improve prediction accuracy.

```
Epoch 292/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 906323886080.0000 - mean_absolute_error: 545619.1250 - val_loss: 882440208384.0000 - val_mean_a
bsolute_error: 543294.3750
Epoch 293/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 869948456960.0000 - mean_absolute_error: 535445.8750 - val_loss: 878965686272.0000 - val_mean_a
bsolute_error: 543887.3750
Epoch 294/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 867647094784.0000 - mean_absolute_error: 537218.5625 - val_loss: 895165005824.0000 - val_mean_a
bsolute_error: 504257.8750
Epoch 295/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 858338754560.0000 - mean_absolute_error: 530790.8125 - val_loss: 870183796736.0000 - val_mean_a
bsolute_error: 529796.1875
Epoch 296/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 873654910976.0000 - mean_absolute_error: 539874.1875 - val_loss: 963666051072.0000 - val_mean_a
bsolute_error: 499296.5938
Epoch 297/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 884484472832.0000 - mean_absolute_error: 537175.0625 - val_loss: 866469675008.0000 - val_mean_a
bsolute_error: 534663.0625
Epoch 298/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 871160479744.0000 - mean_absolute_error: 536198.1250 - val_loss: 1034173677568.0000 - val_mean_
absolute_error: 500695.7500
Epoch 299/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 857545310208.0000 - mean_absolute_error: 532336.1875 - val_loss: 922741506048.0000 - val_mean_a
bsolute_error: 559237.3125
Epoch 300/300
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 854343352320.0000 - mean_absolute_error: 530425.3125 - val_loss: 881796775936.0000 - val_mean_a
bsolute_error: 515203.1562
```

Deeper Network

A deeper network has more neurons, more hidden layers, and has greater capacity to capture complex patterns in the data (Deep Neural Network, n.d.). Now we will use this structure:

```python
model = Sequential()

# Input layer + 1st hidden layer
model.add(Dense(20, activation='relu', input_dim=X_train.shape[1]))


model.add(Dense(20, activation='relu'))
model.add(Dense(50, activation='relu'))
model.add(Dense(50, activation='relu'))
model.add(Dense(100, activation='relu'))
model.add(Dense(100, activation='relu'))
model.add(Dense(200, activation='relu'))
model.add(Dense(200, activation='relu'))
model.add(Dense(100, activation='relu'))
model.add(Dense(100, activation='relu'))
model.add(Dense(50, activation='relu'))
model.add(Dense(50, activation='relu'))
model.add(Dense(20, activation='relu'))
model.add(Dense(10, activation='relu'))

# Output layer (1 neuron for regression)
model.add(Dense(1))

# Compile model
model.compile(
    optimizer='adam',
    loss='mean_squared_error',  # regression loss
    metrics=['mean_absolute_error']  # meaningful metric for regression
)

# Train the model
history = model.fit(X_train, y_train, epochs=1000, validation_split=0.2)
```

Number of Layers

- Input layer: 1
- Hidden layers: 14
- Output layer: 1

Neurons in Each Hidden Layer

1. 20 neurons
2. 20 neurons

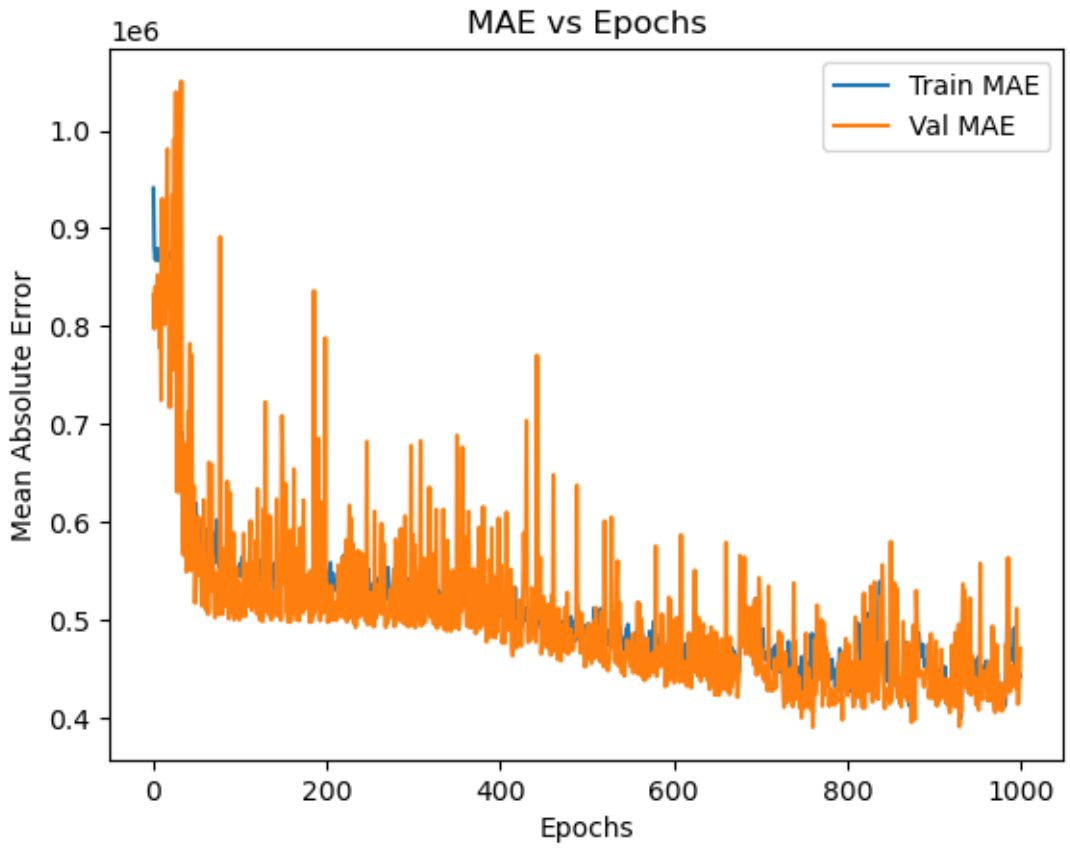
3. 50 neurons
4. 50 neurons
5. 100 neurons
6. 100 neurons
7. 200 neurons
8. 200 neurons
9. 100 neurons
10. 100 neurons
11. 50 neurons
12. 50 neurons
13. 20 neurons
14. 10 neurons

This is quite a deeper neural network with more neurons in each layer, and hence greater probability to achieve better results.

Enhancing the network by adding layers and neurons lowers MAE to approximately 442,000.

```
Epoch 991/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 661020278784.0000 - mean_absolute_error: 459404.3438 - val_loss: 621733150720.0000 - val_mean_a
bsolute_error: 436051.0000
Epoch 992/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 674228469760.0000 - mean_absolute_error: 459668.4688 - val_loss: 662089302016.0000 - val_mean_a
bsolute_error: 435833.0312
Epoch 993/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 741502025728.0000 - mean_absolute_error: 492054.7812 - val_loss: 586751213568.0000 - val_mean_a
bsolute_error: 444024.0625
Epoch 994/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 589669072896.0000 - mean_absolute_error: 442282.0312 - val_loss: 631933501440.0000 - val_mean_a
bsolute_error: 438714.9062
Epoch 995/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 604789145600.0000 - mean_absolute_error: 439543.7500 - val_loss: 589889536000.0000 - val_mean_a
bsolute_error: 430962.2812
Epoch 996/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 614685278208.0000 - mean_absolute_error: 438662.1250 - val_loss: 698933968896.0000 - val_mean_a
bsolute_error: 511801.5625
Epoch 997/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 662223323136.0000 - mean_absolute_error: 462521.5000 - val_loss: 553595109376.0000 - val_mean_a
bsolute_error: 414445.2500
Epoch 998/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 619149000704.0000 - mean_absolute_error: 447149.3125 - val_loss: 609280131072.0000 - val_mean_a
bsolute_error: 414860.7812
Epoch 999/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 620841271296.0000 - mean_absolute_error: 446963.9688 - val_loss: 612707532800.0000 - val_mean_a
bsolute_error: 437220.0312
Epoch 1000/1000
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 639627034624.0000 - mean_absolute_error: 442481.8125 - val_loss: 604519858176.0000 - val_mean_a
bsolute_error: 471066.9062
```

Epochs and MAE graph:



The results highlight that backpropagation is the main driver of learning. More epochs allow repeated refinement of weights. Deeper network architecture enables modeling of complex patterns. These factors lead to better predictions.

Conclusion

Backpropagation plays an important role in improving the performance of neural networks. It is the process that allows the network to learn from its mistakes and become more accurate over time. These small changes help the network make better predictions in the next step. This process of backpropagation is repeated many times during training.

In the diabetes dataset, accuracy increased significantly as the model trained longer. In the car price dataset, the error (MAE) decreased with more training. This clearly shows that backpropagation helps model learn better patterns from the data. It continuously corrects errors, updates the model, and makes predictions more accurate.

References

Backpropagation in Neural Network. (2026). Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/

baeldung. (2025). *Epoch in Neural Networks*. Retrieved from baeldung: https://www.baeldung.com/cs/epoch-neural-networks

Brownlee, J. (2021). *What Is a Gradient in Machine Learning?* Retrieved from machinelearningmastery: https://machinelearningmastery.com/gradient-in-machine-learning/

Chain Rule & Backpropagation. (n.d.). Retrieved from aiunderthehood: https://aiunderthehood.com/ml-essentials/maths-for-aiml/calculus/chain-rule-backpropagation/

Deep Neural Network. (n.d.). Retrieved from sciencedirect: https://www.sciencedirect.com/topics/computer-science/deep-neural-network

Divakar, V. (2024). *Forward Propagation In Neural Networks: Components and Applications*. Retrieved from quantinsti: Forward Propagation In Neural Networks: Components and Applications

Lee, F. (n.d.). *What is a neural network?* Retrieved from IBM: https://www.ibm.com/think/topics/neural-networks

Lee, F. (n.d.). *What is logistic regression?* Retrieved from IBM: https://www.ibm.com/think/topics/logistic-regressio

Luca, G. D. (2025). *Difference Between a Feature and a Label*. Retrieved from baeldung: https://www.baeldung.com/cs/feature-vs-label

Mean Absolute Error In Machine Learning: What You Need To Know. (2023). Retrieved from arize: https://arize.com/blog-course/mean-absolute-error-in-machine-learning-what-you-need-to-know/

Salve, S. (2024). *Forward and Backward Propagation Explained*. Retrieved from Medium: Forward and Backward Propagation Explained

Three-Layered Neural Network. (n.d.). Retrieved from sciencedirect: https://www.sciencedirect.com/topics/engineering/three-layered-neural-network

Weights and Bias in Neural Networks. (2025). Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/deep-learning/the-role-of-weights-and-bias-in-neural-networks/

Yathish, V. (2022). *loss function and there use in neural networks*. Retrieved from towardsdatascience: https://towardsdatascience.com/loss-functions-and-their-use-in-neural-networks-a470e703f1e9/